

Proyecto “Librería”

Explicación Teórica del Funcionamiento de Django y su Integración con Bases de Datos

Autora: Catalina Monserrat Villegas Ortega

Bootcamp Talento Digital – Módulo 7: Django y Bases de Datos

Año: 2025

1. Integración del framework Django con bases de datos

Django utiliza un componente interno llamado **ORM (Object-Relational Mapper)**, que permite interactuar con una base de datos **utilizando objetos de Python en lugar de sentencias SQL**. El ORM traduce las operaciones realizadas sobre los modelos (crear, leer, actualizar, eliminar) en sentencias SQL automáticas, lo que simplifica el desarrollo y permite independencia del motor de base de datos.

◆ Principales características:

- **Abstracción del SQL:** no se requiere escribir consultas SQL manualmente.
- **Independencia del motor:** el mismo código funciona con PostgreSQL, MySQL, SQLite u Oracle.
- **Seguridad:** evita inyecciones SQL mediante consultas parametrizadas.
- **Gestión automática de migraciones:** refleja cambios en los modelos directamente en la base de datos.
- **Escalabilidad:** permite manejar múltiples conexiones y modelos complejos sin perder eficiencia.

2. Configuración y conexión con diferentes motores de base de datos

La conexión se define en el archivo `settings.py`, dentro del diccionario `DATABASES`. Django permite cambiar de motor de base de datos simplemente ajustando el parámetro `ENGINE`.

Ejemplo de configuración — PostgreSQL (usado en este proyecto)

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'libreria_db',
        'USER': 'postgres',
        'PASSWORD': 'tu_contraseña',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

Otros motores compatibles

SQLite (modo desarrollo):

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

MySQL (modo alternativo):

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'libreria_db',
        'USER': 'root',
        'PASSWORD': 'tu_contraseña',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

Gracias a esta estructura modular, **Django se adapta fácilmente a distintos entornos** (desarrollo, pruebas o producción) sin modificar los modelos de datos.

3. Manejo de conexiones y operaciones a través del ORM

El ORM de Django **administra automáticamente las conexiones** a la base de datos según las consultas realizadas.

Cada modelo se traduce en una tabla SQL, y cada instancia de modelo representa una fila de esa tabla.

Ejemplo práctico:

ORM en Python

Libro.objects.filter(precio__gte=20000)

Internamente Django genera la siguiente consulta SQL:

```
SELECT * FROM biblioteca_libro WHERE precio >= 20000;
```

El ORM también maneja:

- **Migraciones automáticas:** con makemigrations y migrate para crear o modificar tablas.
- **Transacciones:** asegura integridad y rollback en caso de error.

- **Agregaciones:** con funciones como Sum(), Avg(), Count(), Min(), Max().
- **Relaciones:** gestiona automáticamente claves foráneas y relaciones complejas (1–1, 1–N, N–N).

4. Aplicaciones preinstaladas en Django

Django incluye una serie de **aplicaciones integradas** que facilitan la construcción de proyectos web completos.

Estas aplicaciones ofrecen funcionalidades esenciales listas para usar, reduciendo el tiempo de desarrollo.

Aplicación	Función principal	Uso en el proyecto “Librería”
django.contrib.admin	Panel de administración	Permite gestionar libros, autores, pedidos y clientes desde una interfaz gráfica.
django.contrib.auth	Sistema de autenticación	Gestiona el registro, login, logout y control de permisos de los usuarios.
django.contrib.sessions	Manejo de sesiones	Almacena información temporal de usuarios autenticados.
django.contrib.messages	Sistema de alertas	Permite mostrar mensajes como “Libro agregado exitosamente”.
django.contrib.staticfiles	Gestión de archivos estáticos	Administra archivos CSS, JS e imágenes en el proyecto.

Ejemplo aplicado

En el proyecto “Librería”:

- django.contrib.admin se usó para registrar los modelos y gestionar datos desde el panel /admin/.
- django.contrib.auth se integró con el decorador @login_required para restringir el acceso a ciertas vistas (como agregar, editar o eliminar libros).
- django.contrib.messages muestra confirmaciones al crear o eliminar registros.
- django.contrib.sessions mantiene activa la sesión del usuario logueado durante su navegación.

5. Conclusión

El proyecto “Librería” demuestra cómo Django facilita el desarrollo web profesional mediante su arquitectura **MVC** y el uso eficiente del **ORM**. La integración con **PostgreSQL** permite un control completo sobre los datos, mientras que las

aplicaciones preinstaladas proporcionan herramientas robustas para autenticación, administración y manejo de sesiones.

En conjunto, este ecosistema permitió crear una aplicación **segura, modular, escalable y mantenible**, cumpliendo todos los requerimientos del portafolio del Módulo 7.